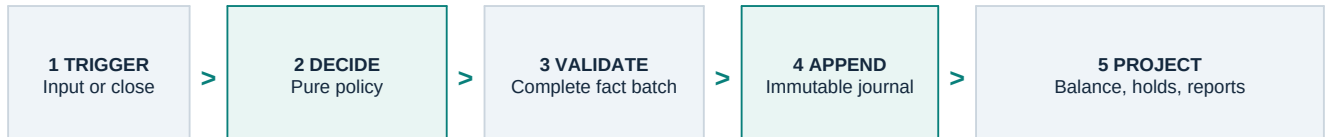


Architecture & Trade-offs

Follow the money. Keep the history. Know what changes in production.

What exists today

This core has one job: receive the supplied events, record what happened, and calculate balances without changing history. It runs entirely in memory because the assessment excludes databases, services, and user interfaces.



An event is a request such as a credit, debit, authorization, settlement, or reversal. Policy decides whether the request is valid and what it should produce. The journal appends the complete result as one batch. Reports derive balances and authorization states from the stored facts.

Three guarantees organize the implementation:

- **Exact money:** AED is stored in hundredths and BHD in thousandths. Currency arithmetic uses integers, never binary floating point.
- **Permanent history:** an accepted fact is never edited or deleted. A correction adds a linked opposite fact.
- **All or nothing:** a batch is returned only when every fact and relationship in it is valid.

"All or nothing" currently means one in-process function call. It does not mean the data survives a crash, and it does not protect two processes writing at the same time. Those are production guarantees, not claims made by this implementation.

The boundary I chose

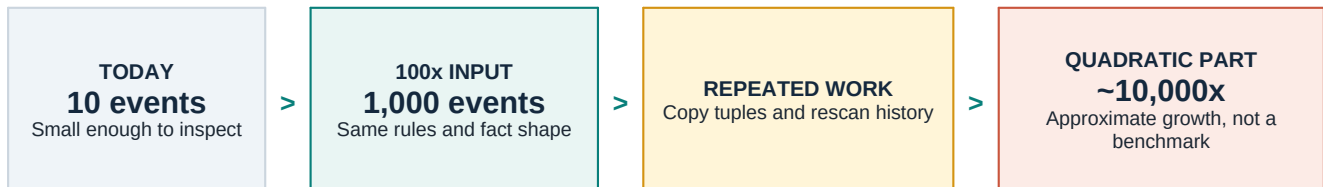
Business rules live in policy and orchestration. The journal checks that the facts produced by those rules fit together: currency matches the account, direct postings match their event, authorization transitions match their request, settlements consume an active hold only once, and reversals point to one exact debit.

This is a **customer-account subledger**. A positive posting increases the customer's displayed balance and a negative posting reduces it. It is not the bank's complete accounting book because the prompt gives only the customer side of each movement. The missing side is the largest deliberate cut in this design, explained on page 4.

Append-only at 100x

Append-only is useful because the story remains explainable, but the current data structure pays for that clarity with repeated work. The ledger is an immutable tuple. Each append copies the tuple, several validations scan earlier facts, and replay can retain previous ledger versions.

The supplied stream has ten events. At 100 times that input, the core would process about 1,000 events. Because repeated copying and retained prefixes can grow roughly with the square of the event count, that part of the work can become about 10,000 times larger. This is an explanation of the algorithm, not a throughput benchmark.



Two failures, not one

It becomes slow and memory-hungry. The core repeatedly walks and copies history that did not change.

It has no official winner between concurrent writes. Two writers can read the same version, both approve their event, and both create a different successor with the same sequence number. Both results can look valid in isolation.

Cheapest change first

I would first stop retaining full snapshots after every replay step. Keep one current ledger, record only the newly appended range for each step, and add indexes for frequent lookups such as account postings, authorizations, fees, and reversals. This improves the current design without changing its business behavior.

The next production step is one transactional database writer:

- append only when the expected journal sequence still matches;
- enforce unique event and record identities in the database;
- store the event result and its pending outbound message in the same transaction;
- build balance checkpoints and indexes from measured queries; and
- prove backup restoration before claiming durability.

I would not start with Kafka, microservices, sharding, or multi-region writes. Those tools introduce new ordering and recovery problems before this workload proves that one database writer is insufficient. If later measurements require partitioning, one ordered owner per account is the natural next boundary. A transfer between two accounts would still need a separate guarantee that both sides succeed or both fail.

Value-dated entries in production

An entry has two dates and one sequence. **Booking date** says when the bank recorded it. **Value date** says when its financial effect applies. **Journal sequence** says where it sits in the append order.

BOOKING DATE When the bank records it	VALUE DATE When the financial effect applies	JOURNAL SEQUENCE Its exact append order
APPEND NOW > earlier value date > recalculated past result > linked replacement		

A late entry is appended now but can carry an earlier value date. No existing record changes. A recalculated past balance may change because it includes one more fact, which can affect fees, interest, statements, reconciliation, and downstream data. The previously issued result remains stored; a correction creates a linked replacement.

I assume "UAE-licensed bank" means an institution licensed and supervised by the CBUAE. Its consumer standards require statements to show opening and closing balances, deposits, withdrawals, interest or profit, and fees,¹ with accurate disclosures and calculations.² UAE AML/CFT rules also require records sufficient to reconstruct individual transactions.³ These rules do not prescribe this architecture, but they make untracked corrections risky.

One control before go-live: protect closed-period results

The current core rejects an entry whose value date falls inside a finalized period. Before handling real customer money, I would keep the normal path blocked and send late entries to a separate correction workflow. It must:

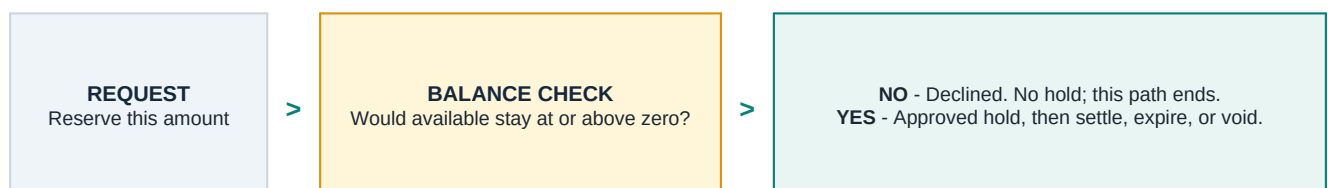
- hold the entry instead of posting it;
- calculate and show everything it would change;
- preserve old outputs and create linked replacements; and
- save the correction and all replacements together, or save nothing.

Until those steps succeed, the issued result stays unchanged. This prevents an untracked correction. An approval step may be added, but it does not replace the enforced rule.

Authorization lifecycle

An authorization is a reservation, not a debit. It reduces the amount still available to spend. Settlement is the later event that actually removes money and releases any unused reservation.

The current model supports approval, decline, and one final settlement. A decline is already final because it creates no hold. Once a hold is approved, however, this model has no ending other than settlement. A production system must represent the other endings explicitly:



- **Expire:** no charge arrives before the deadline; release the hold once.
- **Void or reduce:** the purchase is cancelled, duplicated, or becomes cheaper; authenticate the instruction, record the reason, and release the exact amount.
- **Unknown result:** the caller loses the response while settlement may have succeeded; look up and reconcile the stored result before retrying or releasing anything.
- **Refund or chargeback:** money has already settled and must move back; append a new linked credit instead of reopening the old hold.

What I cut and why

The biggest cut: this is not double-entry

The current core records one customer-facing effect. A real double-entry transaction records at least one debit and one credit, and the totals must be equal. For a bank, a customer's deposit is a liability: the bank owes that money to the customer.⁴

The deposit example below is conceptual. It shows the missing accounting shape; it does not claim that the assessment supplied a real cash or settlement account.

CONCEPTUAL BANK VIEW - CUSTOMER DEPOSITS AED 1,200		
DEBIT AED 1,200 Cash / settlement asset What the bank controls	=	CREDIT AED 1,200 Customer deposit liability What the bank owes
TOTAL DEBITS AED 1,200 = TOTAL CREDITS AED 1,200		
Both legs share one transaction identity and commit together. The counterpart shown here is illustrative.		

What "double-entry" must mean

- every transaction contains at least one debit and one credit;
- total debits equal total credits inside that transaction;
- balancing happens separately for each currency;
- every leg names a real account and uses an explicit debit or credit side; and
- all legs commit together, so half a movement is never visible.

Those are the accounting guarantees. Production safety adds immutable posted entries, linked corrections, unique request identities, ordered writes, audit evidence, period controls, reconciliation, backups, and tested recovery. Modern ledger implementations commonly group two or more entries into one atomic balanced transaction and distinguish debit-normal from credit-normal accounts.⁵

Other deliberate cuts

- **No persistence:** fine for a six-day replay; production needs a durable append and restore tests.
- **No FX:** the supplied accounts never exchange currencies; production FX needs a sourced rate, quote time, rounding rule, and gain or loss treatment.
- **Integer days:** deterministic here; production needs real dates and timestamps, business calendars, cut-off rules, and explicit period status.
- **One final authorization capture:** enough here; production needs tested transitions for expiry, change, partial capture, dispute, and refund.
- **Trusted function calls:** authentication is out of scope; production needs actor identity, authorization, audit evidence, and controlled delivery.
- **One global sequence:** fine here; production must measure its limit before adding ordered account ownership without breaking transfers.

I would first make one ledger balanced, durable, recoverable, controlled, and reconcilable. I would distribute it only after measurements show which limit needs it. The tests prove the six-day behavior, not a full production bank ledger.

Sources and regulatory context

1. [CBUAE Consumer Protection Standards, Article 2, clause 2.1.2.9](#)
2. [CBUAE Consumer Protection Regulation, Article 2, clause 2.1.2.10](#)
3. [UAE Cabinet Resolution No. 134 of 2025, Article 25](#)
4. [Bank of England, customer deposits as bank liabilities](#)
5. [Modern Treasury, debits, credits, account normality, and balanced transactions](#)